# Project Plan: The Verified Market Simulator Pipeline

Financial Computing & Algo Trading Section

Spring 2026

## 1 Mission & Architecture

The goal is to build a high-performance, purely functional **Market Simulation Pipeline**. Unlike monolithic simulators, this system is designed as a modular set of tools that allow researchers to "Bring Your Own Data" (BYOD), ensuring compliance with LOBSTER data licensing while providing a rigorous foundation for academic research.

## 2 Target Use Cases

This infrastructure is designed to solve specific problems for current teaching and research modules within the department:

### 2.1 1. Market Microstructure (Sylvia)

- **Need:** Visualising the hidden dynamics of the Limit Order Book (LOB) to understand liquidity and price formation.
- **Deliverable:** The "Visualiser Workstream" (Phase 3) will provide tools to take differential snapshots of the book state, allowing students to visually replay market events and verify theoretical models against LOBSTER data.

### 2.2 2. Algorithmic Trading (Paolo & Nick)

- **Need:** A reliable, high-speed engine to backtest trading strategies without the risk of "look-ahead bias" or state corruption common in ad-hoc simulators.
- **Deliverable:** The "Simulation Pipeline" (Phase 2) will allow students to plug in OCaml strategy modules and run parallel backtests on the HPC cluster, receiving accurate PnL and execution metrics.

### 2.3 3. PhD & MSc Research Infrastructure

- **Current State:** PhD students (e.g., Konark Jain, Antonio Briola) have previously been forced to build ad-hoc Python matching engines. While functional, these lack formal correctness guarantees and are hard to reuse.
- **Future State:** This project formalizes that ad-hoc work into a **Verified Infrastructure**. Future PhD projects can skip the "engine building" phase and focus immediately on signal construction and formal verification of execution logic, trusting the underlying matching engine to be bug-free.

## 3 The Pipeline Architecture

The system consists of two distinct, decoupled pipelines to balance performance with licensing constraints:

1. **The Data Pipeline (Preprocessing):** A local tool that ingests raw LOBSTER CSVs (provided by the user) and converts them into a high-efficiency, compressed binary format optimised for HPC storage and streaming.
2. **The Simulation Pipeline (Backtesting):** A purely functional OCaml engine running on the HPC cluster that streams the binary data, rebuilds the Limit Order Book (LOB) without mutation, and executes strategy logic.

## 4 Team Structure (Flexible)

We anticipate a team of 3–4 students. Rather than rigid silos, we propose a **Collaborative Swarm** model where all members participate in core development and verification, with rotating leads for specific components.

- **All Members:** Contribute to the core OCaml matching logic and Property-Based Testing (QCheck). This ensures everyone gains exposure to Functional Programming and Verification.
- **Rotation A (Infrastructure):** Focuses on the HPC Data Pipeline, binary file I/O, and parallelization.
- **Rotation B (Theory):** Focuses on formalising invariants (e.g., Fair Matching) and proving them using formal methods (Coq/Cameleer).

## 5 Execution Timeline (6 Months)

### Phase 1: The "Quick Start" & Data Foundation (Weeks 1–6)

*Goal: Get a walking skeleton running immediately.*

- **Team Task:** Complete *Real World OCaml* (Ch 1–5) and set up HPC environments.
- **The Data Pipeline:**
  - Build the *Python Preprocessor* (using Polars) to convert CSV → Binary locally.
  - Define the *Binary Schema*: Fixed-width integers for Price/Size/Time to avoid floating-point drift.
- **The Engine:**
  - Implement the `OrderBook` data structure (Map/Tree-based).
  - Implement a basic `CsvReader` (temporary) to allow immediate testing of the engine before the full Binary pipeline is ready.

### Phase 2: The Core Matching Engine (Weeks 7–14)

*Goal: A correct, functional LOB implementation.*

- **Core Logic:** Implement the immutable `match` function.
  - Logic: $match(Book_t, Order_{new}) \rightarrow (Book_{t+1}, Trades, Updates)$.
- **Verification (All Hands):**
  - Define Invariants: *Best Bid* $<$ *Best Ask*, *Trade Price* $\in$ *[Limit, Market]*.
  - Implement **QCheck** generators to fuzz-test the engine with millions of random orders.
- **HPC Integration:** Connect the OCaml engine to the *Binary Data Stream* created in Phase 1.

### Phase 3: Formal Verification & Visualization (Weeks 15–24)

*Goal: Prove correctness and visualise drift.*

- **The "Verifier" Workstream:**
  - Map the OCaml types to a Theorem Prover.
  - Attempt to prove that the matching logic satisfies *Fairness* (Price-Time Priority) and *Individual Rationality*.
- **The "Visualiser" Workstream:**
  - Build a tool to compare two state snapshots ($Book_t$ vs $Book'_t$) to visually debug any state drift.
  - Support the visualisation requirements for the Market Microstructure class.
- **Performance Tuning:** Profile the engine on the HPC cluster and optimise Garbage Collection (GC) for high throughput.

## 6 Why This Approach?

- **Flexibility:** By decoupling Data from Simulation, we avoid licensing issues and allow the engine to be open-sourced while data remains private.
- **Rigour :** Every team member gains exposure to Formal Verification, a highly sought-after skill in modern systems engineering (e.g., AWS, Jane Street).
- **Efficiency:** The binary pipeline ensures that the HPC cluster spends CPU cycles on *matching*, not text parsing.

## 7 Further Plans

- **LOB Visualiser** - Graphical snapshots and animations, other means of visually characterising order-flow.
- **Algo Trading Backtester** - Possibility of wrapping Python or other Code into OCaml and backtesting performance.

## References


[1] Suneel Sarswat and Abhishek Kr Singh. *Formally Verified Trades in Financial Markets*. In *Formal Methods and Software Engineering (ICFEM 2020)*, pages 217–232. Springer, 2020.

[2] Yaron Minsky and Stephen Weeks. *Caml trading: experiences with functional programming on Wall Street*. Journal of Functional Programming, 18(4):553–564, 2008.